

计算机设计与实践

流水线CPU及SoC设计

任务A：数据前递

2026·夏



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验目的

- ◆ 理解数据前递解决流水线CPU数据冒险的原理
- ◆ 掌握数据前递的实现方法
- ◆ 熟练掌握数字电路的仿真调试方法



实验内容

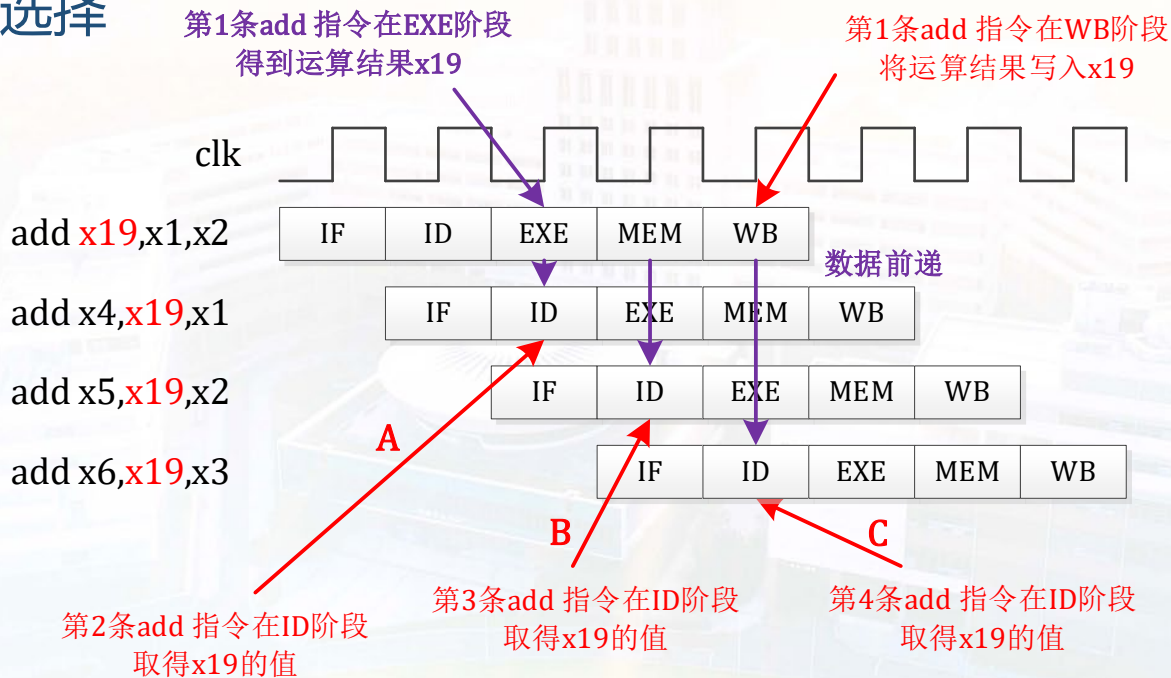
- ◆ 设计实现流水线CPU的数据前递机制，解决数据冒险
- ◆ 通过功能仿真验证流水线CPU的数据前递机制



数据前递

◆ 数据前递

将**EXE**、**MEM**、**WB**阶段的数据前递到**ID**阶段，使其参与译码运算的源操作数选择



数据前递

◆ 实现要点

① 前递的数据来源

wdata_ex: 执行阶段的结果

wdata_mem: 访存阶段的结果

wdata_wb: 写回阶段的结果

② 根据冲突类型选择前递数据

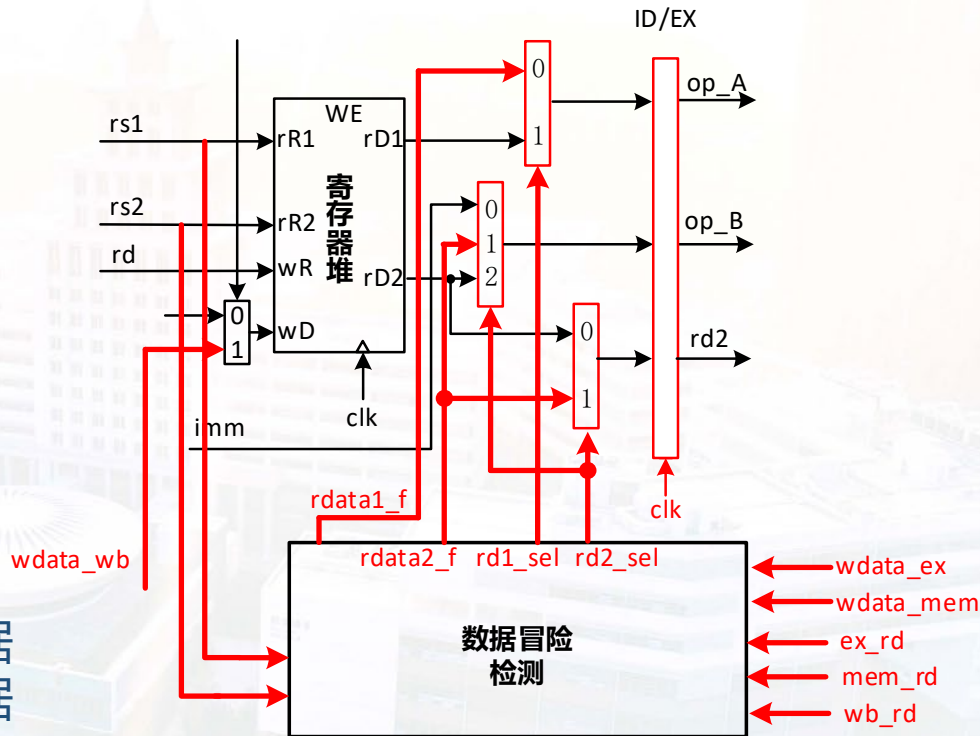
rdata1_f: rdata1对应的前递数据

rdata2_f: rdata2对应的前递数据

③ 增加数据选择信号

rd1_sel: rdata1是否选择前递数据

rd2_sel: rdata2是否选择前递数据



数据前递

◆ 实现示例（写回与译码阶段冲突）

RTL实现

```
wire rs1_id_wb_con = (raddr1 == waddr) & we & re1;
```

```
wire rs2_id_wb_con = (raddr2 == waddr) & we & re2;
```

```
assign rdata1 = rs1_id_wb_con ? wdata : re1 ? rf[raddr1] : 'h0;
```

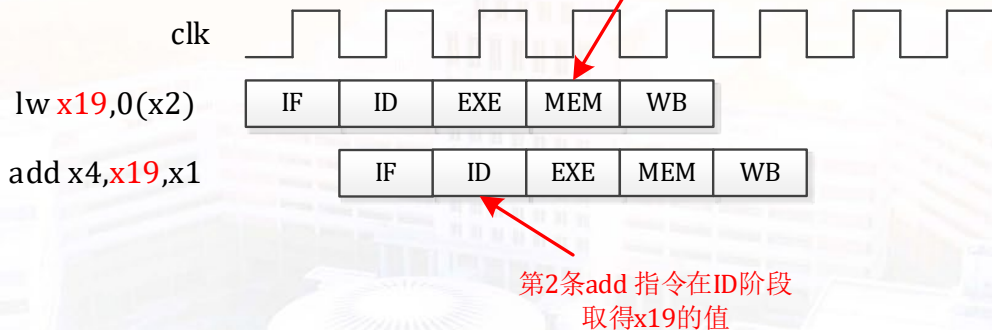
```
assign rdata2 = rs2_id_wb_con ? wdata : re2 ? rf[raddr2] : 'h0;
```

数据前递

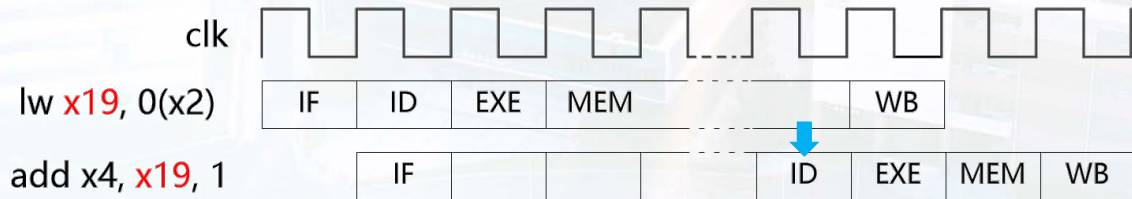
◆ Load-Use冒险

➤ Load-Use冒险是数据冒险情形A的特例

第1条add指令在MEM阶段
取回对应地址的值



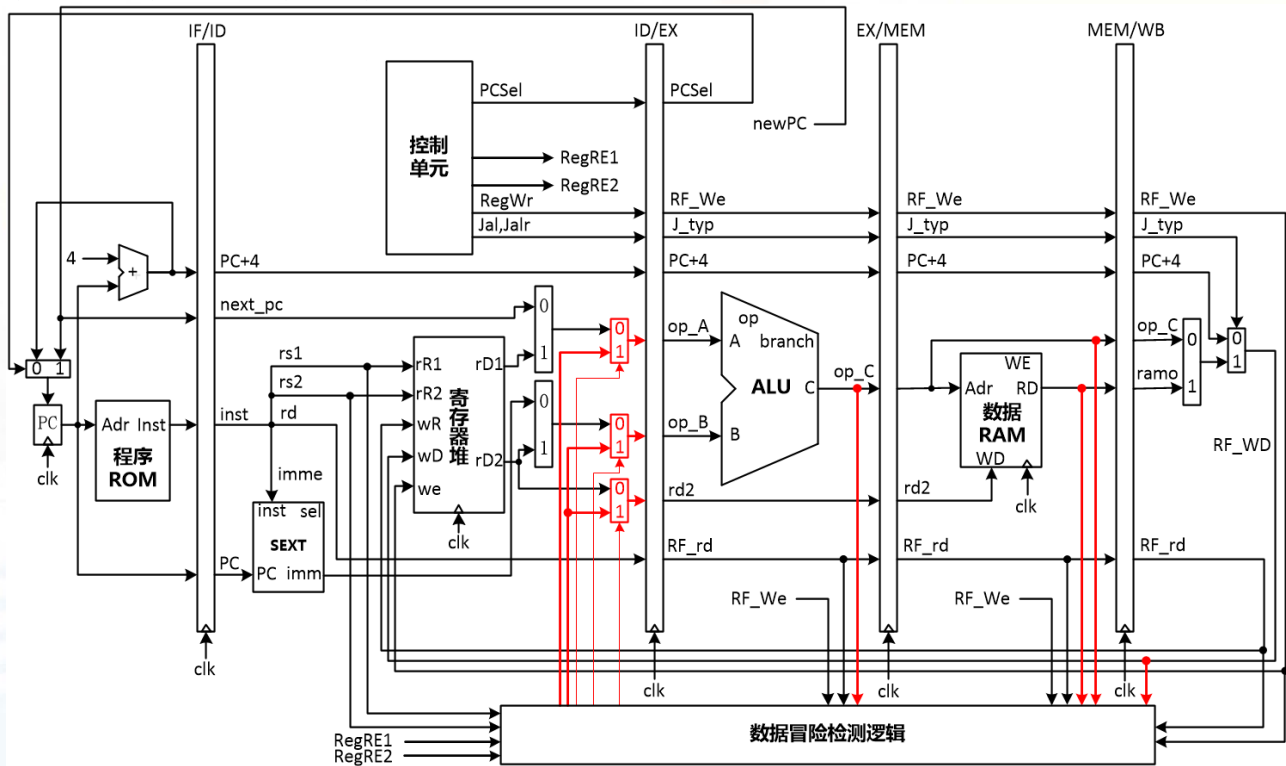
➤ 基本思路：先通过暂停，令Load指令取到数据，再从MEM前递数据到ID



流水线数据通路

- 增加数据前递的数据通路

示例



数据前递仿真

- ◆ 在Vivado中使用默认的lw.coe程序运行功能仿真
- ◆ 观察波形，分析流水线冒险是否能正确检测，并且流水线CPU能否正确前递数据，最终相关指令能否正确写回
- ◆ 通过功能仿真后，可尝试使用Trace框架对CPU进行功能验证
- ◆ **注意代码规范性，可以避免很多问题！**

AXI Trace注意事项

◆ 流水线CPU需自行连接debug信号

```
01 wire [31:0] debug_wb_pc    ;    // WB阶段的PC
02 wire          debug_wb_rf_we;    // WB阶段的寄存器写使能
03 wire [ 4:0] debug_wb_rf_wR;    // WB阶段的目标寄存器 (若wb_rf_we为0, 此项可为任意值)
04 wire [31:0] debug_wb_rf_wD;    // WB阶段写入寄存器的值 (若wb_rf_we为0, 此项可为任意值)
05
06 wire [31:0] debug_mem_pc    ;    // MEM阶段的PC
07 wire [ 3:0] debug_mem_we    ;    // MEM阶段写访存时的写使能
08 wire [31:0] debug_mem_waddr;    // MEM阶段写访存时的写地址 (若mem_we为0, 此项可为任意值)
09 wire [31:0] debug_mem_wdata;    // MEM阶段写访存时的写数据 (若mem_we为0, 此项可为任意值)
```

- ◆ 哪个流水线阶段发出访存请求，就把哪个阶段的信号连接到debug_mem_*
- ◆ 哪个阶段写回寄存器，就把哪个阶段的信号连接到debug_wb_*



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ